

PREMIER UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



PROJECT REPORT

COURSE CODE : CSE 4346

COURSE TITLE : BIG DATA ANALYTICS LABORATORY.

REPORT NAME : Household Electric Power Consumption Analysis

DATE : 15-09-2026

SUBMITTED TO

MD TAMIM HOSSAIN

Lecturer

Department of CSE
Premier University Chittagong

SUBMITTED BY

NAME : SYED INTIQAB AHMED

ID : 0222220005101167

SEMESTER : 8th

SECTION : C **SESSION** : SPRING 2026

TABLE OF CONTENTS

Topic Description	Page
Problem Statement	2
Dataset Description	2
Summary of Data Analysis	3
MapReduce Analysis	4
Maximum Power Consumption	4
Average Voltage	6
Total Energy Consumption	8
Pig Electric Power Data Analysis	10
Top 10 Power Consumption	10
Maximum Voltage Values	11
Average Consumption	12
High Power Filtering	13
Sort by Power Consumption	14
Conclusion	15

1. Problem Statement & Dataset Schema Description

Problem Statement

Analyze the Household Electric Power Consumption dataset using Hadoop MapReduce (Java) and Apache Pig Latin. The objective is to perform distributed aggregations, filtering, sorting, and extraction of power usage patterns, peak consumption levels, total energy outputs, and average electrical metrics across time series data.

Dataset Schema Description

Dataset Source: Household Electric Power Consumption Repository
<https://archive.ics.uci.edu/dataset/235/individual%2Bhousehold%2Belectric%2Bpower%2Bconsumption>

Field Name	Data Type	Description
Date	chararray (String)	Date in format dd/mm/yyyy
Time	chararray (String)	Time in format hh:mm:ss
Global_active_power	double	Household global minute-averaged active power (in kW)
Global_reactive_power	double	Household global minute-averaged reactive power (in kW)
Voltage	double	Minute-averaged voltage (in Volts)
Global_intensity	double	Household global minute-averaged current intensity (in Amperes)

Field Name	Data Type	Description
Sub_metering_1	double	Energy sub-metering No. 1 (in watt-hour of active energy)
Sub_metering_2	double	Energy sub-metering No. 2 (in watt-hour of active energy)
Sub_metering_3	double	Energy sub-metering No. 3 (in watt-hour of active energy)

2. MapReduce Data Analysis (Java)

MapReduce Problem 1: Maximum Power Consumption

Description: Reads the Global_active_power field (3rd column) across all records and determines the single highest active power consumption value.

Source Code (MaxPower.java):

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MaxPower {
    public static class MaxPowerMapper
        extends Mapper<Object, Text, Text, DoubleWritable> {

        private final Text outputKey = new Text("Maximum_Power");
        private final DoubleWritable outputValue = new DoubleWritable();

        public void map(Object key, Text value, Context context)
```

```

        throws IOException, InterruptedException {

        String line = value.toString();

        // Skip header
        if (line.startsWith("Date,Time")) {
            return;
        }

        String[] fields = line.split(",");

        try {
            if (fields.length >= 9) {
                double power = Double.parseDouble(fields[2]);
                outputValue.set(power);
                context.write(outputKey, outputValue);
            }
        } catch (NumberFormatException e) {
            // Ignore invalid records
        }
    }
}

public static class MaxPowerReducer
    extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {

    private double maxPower = Double.MIN_VALUE;

    public void reduce(Text key, Iterable<DoubleWritable> values,
        Context context)
        throws IOException, InterruptedException {

        for (DoubleWritable value : values) {
            if (value.get() > maxPower) {
                maxPower = value.get();
            }
        }

        context.write(key, new DoubleWritable(maxPower));
    }
}

public static void main(String[] args) throws Exception {
    if (args.length != 2) {
        System.err.println("Usage: MaxPower <input path> <output path>");
        System.exit(2);
    }

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Maximum Power Consumption");

```

```

        job.setJarByClass(MaxPower.class);
        job.setMapperClass(MaxPowerMapper.class);
        job.setReducerClass(MaxPowerReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(DoubleWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

```

Execution & Output Verification:

```

D:\PowerConsumption\MapReduce\MaxPower>hadoop jar MaxPower.jar MaxPower /PowerConsumption/input/hous
d_power_consumption.csv /PowerConsumption/output/maxPower
2026-09-13 19:05:23,552 INFO client.RMPProxy: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-13 19:05:24,180 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not p
rmed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-09-13 19:05:24,215 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/
op-yarn/staging/USER/.staging/job_1789303575219_0001
2026-09-13 19:05:24,441 INFO input.FileInputFormat: Total input files to process : 1
2026-09-13 19:05:24,546 INFO mapreduce.JobSubmitter: number of splits:1
2026-09-13 19:05:24,673 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1789303575219_00
2026-09-13 19:05:24,675 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-13 19:05:24,831 INFO conf.Configuration: resource-types.xml not found
2026-09-13 19:05:24,831 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-13 19:05:25,443 INFO impl.YarnClientImpl: Submitted application application_1789303575219_00
2026-09-13 19:05:25,478 INFO mapreduce.Job: The url to track the job: http://DESKTOP-I9MNJ0D:8088/pr
application_1789303575219_0001/
2026-09-13 19:05:25,478 INFO mapreduce.Job: Running job: job_1789303575219_0001
2026-09-13 19:05:36,632 INFO mapreduce.Job: Job job_1789303575219_0001 running in uber mode : false
2026-09-13 19:05:36,632 INFO mapreduce.Job: map 0% reduce 0%
2026-09-13 19:05:45,801 INFO mapreduce.Job: map 100% reduce 0%
2026-09-13 19:05:54,868 INFO mapreduce.Job: map 100% reduce 100%
2026-09-13 19:05:54,868 INFO mapreduce.Job: Job job_1789303575219_0001 completed successfully
2026-09-13 19:05:54,947 INFO mapreduce.Job: Counters: 54
    File System Counters
        FILE: Number of bytes read=49182726
        FILE: Number of bytes written=98842911
        FILE: Number of read operations=0
        FILE: Number of large read operations=0
        Bytes Written=21

D:\PowerConsumption\MapReduce\MaxPower>hdfs dfs -cat /PowerConsumption/output/maxPower/part-r-00000
Maximum_Power 11.122

D:\PowerConsumption\MapReduce\MaxPower>_

```

MapReduce Problem 2: Average Voltage

Description: Extracts the Voltage field (5th column) across all dataset records to calculate the overall mean voltage level.

Source Code (AverageVoltage.java):

```

import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

```

```

import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class AverageVoltage {
    public static class VoltageMapper
        extends Mapper<LongWritable, Text, Text, DoubleWritable> {

        private static final Text KEY = new Text("Average_Voltage");
        private DoubleWritable voltage = new DoubleWritable();

        public void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {

            String line = value.toString();

            // Skip header
            if (line.startsWith("Date,")) {
                return;
            }

            String[] fields = line.split(",");

            // Voltage is the 5th column
            if (fields.length >= 5) {
                try {
                    double v = Double.parseDouble(fields[4]);
                    voltage.set(v);
                    context.write(KEY, voltage);
                } catch (NumberFormatException e) {
                    // Ignore invalid records
                }
            }
        }

        public static class VoltageReducer
            extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {

            private DoubleWritable result = new DoubleWritable();

            public void reduce(Text key, Iterable<DoubleWritable> values,
                Context context)
                throws IOException, InterruptedException {

                double sum = 0.0;

```

```

        long count = 0;

        for (DoubleWritable value : values) {
            sum += value.get();
            count++;
        }

        if (count > 0) {
            double average = sum / count;
            result.set(average);
            context.write(key, result);
        }
    }
}

public static void main(String[] args) throws Exception {
    if (args.length != 2) {
        System.err.println("Usage: AverageVoltage <input path> <output path>");
        System.exit(-1);
    }

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Average Voltage");

    job.setJarByClass(AverageVoltage.class);
    job.setMapperClass(VoltageMapper.class);
    job.setReducerClass(VoltageReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(DoubleWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Execution & Output Verification:


```

2026-09-13 19:19:15,975 INFO mapreduce.Job: The url to track the job: http://localhost:19864/job/application_1789303575219_0002/
2026-09-13 19:19:15,383 INFO mapreduce.Job: Running job: job_1789303575219_0002
2026-09-13 19:19:23,579 INFO mapreduce.Job: Job job_1789303575219_0002 running in uber mode : false
2026-09-13 19:19:23,579 INFO mapreduce.Job: map 0% reduce 0%
2026-09-13 19:19:32,757 INFO mapreduce.Job: map 100% reduce 0%
2026-09-13 19:19:40,868 INFO mapreduce.Job: map 100% reduce 100%
2026-09-13 19:19:40,868 INFO mapreduce.Job: Job job_1789303575219_0002 completed successfully
2026-09-13 19:19:40,947 INFO mapreduce.Job: Counters: 54
    File System Counters
        FILE: Number of bytes read=53281286
        FILE: Number of bytes written=107040043
        FILE: Number of read operations=0
        FILE: Number of large read operations=0
        FILE: Number of write operations=0
        HDFS: Number of bytes read=110320717
        HDFS: Number of bytes written=35
        HDFS: Number of read operations=8
        HDFS: Number of large read operations=0
        HDFS: Number of write operations=2
        HDFS: Number of bytes read erasure-coded=0
    Job Counters

```

```

D:\PowerConsumption\MapReduce\AverageVoltage>hdfs dfs -cat /PowerConsumption/output/averageVoltage/part-r-00000
Average_Voltage 240.83985797449844
D:\PowerConsumption\MapReduce\AverageVoltage>

```

MapReduce Problem 3: Total Energy Consumption

Description: Aggregates global active power consumption across all records by calculating the sum of the Global_active_power column.

Source Code (TotalEnergy.java):

```

import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class TotalEnergy {
    public static class EnergyMapper
        extends Mapper<LongWritable, Text, Text, DoubleWritable> {

        private static final Text KEY = new Text("Total_Energy");
        private DoubleWritable power = new DoubleWritable();

```

```

        public void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {

            String line = value.toString();

            // Skip header
            if (line.startsWith("Date, ")) {
                return;
            }

            String[] fields = line.split(",");

            // Global_active_power is the 3rd column
            if (fields.length >= 3) {
                try {
                    double activePower = Double.parseDouble(fields[2]);
                    power.set(activePower);
                    context.write(KEY, power);
                } catch (NumberFormatException e) {
                    // Ignore invalid records
                }
            }
        }

    public static class EnergyReducer
        extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {

        private DoubleWritable result = new DoubleWritable();

        public void reduce(Text key, Iterable<DoubleWritable> values,
            Context context)
            throws IOException, InterruptedException {

            double total = 0.0;

            for (DoubleWritable value : values) {
                total += value.get();
            }

            result.set(total);
            context.write(key, result);
        }
    }

    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Usage: TotalEnergy <input path> <output path>");
            System.exit(-1);
        }
    }

```

```

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Total Energy Consumption");

        job.setJarByClass(TotalEnergy.class);
        job.setMapperClass(EnergyMapper.class);
        job.setReducerClass(EnergyReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(DoubleWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }

```

Execution & Output Verification:

```

2026-09-13 20:59:04,604 INFO input.FileInputFormat: Total input files to process : 1
2026-09-13 20:59:04,684 INFO mapreduce.JobSubmitter: number of splits:1
2026-09-13 20:59:04,795 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1789303575219_0003
2026-09-13 20:59:04,797 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-13 20:59:04,963 INFO conf.Configuration: resource-types.xml not found
2026-09-13 20:59:04,963 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-13 20:59:05,030 INFO impl.YarnClientImpl: Submitted application application_1789303575219_0003
2026-09-13 20:59:05,059 INFO mapreduce.Job: The url to track the job: http://DESKTOP-I9MNJ0D:8088/proxy/application_1789303575219_0003/
2026-09-13 20:59:05,059 INFO mapreduce.Job: Running job: job_1789303575219_0003
2026-09-13 20:59:12,222 INFO mapreduce.Job: Job job_1789303575219_0003 running in uber mode : false
2026-09-13 20:59:12,222 INFO mapreduce.Job:  map 0% reduce 0%
2026-09-13 20:59:20,299 INFO mapreduce.Job:  map 100% reduce 0%
2026-09-13 20:59:28,377 INFO mapreduce.Job:  map 100% reduce 100%
2026-09-13 20:59:28,384 INFO mapreduce.Job: Job job_1789303575219_0003 completed successfully
2026-09-13 20:59:28,454 INFO mapreduce.Job: Counters: 54
    File System Counters
      FILE: Number of bytes read=47133446
      FILE: Number of bytes written=94744359
      FILE: Number of read operations=0
      FILE: Number of large read operations=0
      FILE: Number of write operations=0
      HDFS: Number of bytes read=110320717

```

```

D:\PowerConsumption\MapReduce\TotalEnergy>hdfs dfs -cat /PowerConsumption/output/totalEnergy/part-r-00000
Total_Energy      2237024.862000551

D:\PowerConsumption\MapReduce\TotalEnergy>_

```

3. Apache Pig Data Analysis (Pig Latin)

Pig Problem 1: Top 10 Power Consumption

Description: Filters invalid entries, sorts the entire dataset by Global_active_power in descending order, and limits output to the top 10 highest consumption records.

Pig Script:

```
data = LOAD 'file:///D:/PowerConsumption/data/household_power_consumption.csv'
```

```

        USING PigStorage(',')
        AS (
            Date:chararray,
            Time:chararray,
            Global_active_power:double,
            Global_reactive_power:double,
            Voltage:double,
            Global_intensity:double,
            Sub_metering_1:double,
            Sub_metering_2:double,
            Sub_metering_3:double
        );

clean_data = FILTER data BY Global_active_power IS NOT NULL;
sorted_data = ORDER clean_data BY Global_active_power DESC;
top10 = LIMIT sorted_data 10;

STORE top10 INTO 'file:///D:/PowerConsumption/Pig/output/Top10'
USING PigStorage('\t');

```

Execution & Output Verification:

```

C:\PowerConsumption\Pig\Top10Power>type D:\PowerConsumption\Pig\output\Top10\part-r-00000
2/2/2009      17:09:00      11.122  0.174  229.78  48.4   35.0   69.0   16.0
/3/2007       19:34:00       10.67  0.516  230.2   46.4   37.0   69.0   16.0
/3/2007       19:33:00       10.65  0.5    229.97  46.4   36.0   70.0   17.0
2/2/2009      17:08:00       10.536 0.192  230.24  45.8   23.0   69.0   16.0
0/11/2008     20:19:00       10.348 0.084  231.6   44.6   36.0   71.0   16.0
9/10/2008     01:24:00       10.29  0.302  230.9   44.6   35.0   66.0   17.0
7/1/2008      19:24:00       10.162 0.224  229.16  44.2   71.0   0.0    0.0
/3/2007       19:32:00       10.154 0.502  229.72  44.4   36.0   61.0   16.0
0/11/2008     20:17:00       10.074 0.06   231.41  43.4   36.0   70.0   16.0
9/10/2008     01:25:00       10.064 0.324  231.48  43.4   36.0   69.0   17.0

C:\PowerConsumption\Pig\Top10Power>

```

Pig Problem 2: Maximum Voltage Values

Description: Groups the non-null voltage records across the dataset and computes the maximum voltage value achieved.

Pig Script:

```

data = LOAD 'file:///D:/PowerConsumption/data/household_power_consumption.csv'
        USING PigStorage(',')
        AS (
            Date:chararray,
            Time:chararray,
            Global_active_power:double,
            Global_reactive_power:double,
            Voltage:double,
            Global_intensity:double,

```

```

        Sub_metering_1:double,
        Sub_metering_2:double,
        Sub_metering_3:double
    );

clean_data = FILTER data BY Voltage IS NOT NULL;
grouped_data = GROUP clean_data ALL;

maximum_voltage = FOREACH grouped_data GENERATE
    MAX(clean_data.Voltage) AS Maximum_Voltage;

STORE maximum_voltage
INTO 'file:///D:/PowerConsumption/Pig/output/MaximumVoltage'
USING PigStorage('\t');

```

Execution & Output Verification:

```

D:\PowerConsumption\Pig\AverageConsumption>type D:\PowerConsumption\Pig\output\AverageConsumption\part-r-00000
/2      1.03230883156646709
0/      1.1304623842592554
1/      1.4042360780960446
10      1.1398745904809866
11      1.3594035926686139
12      1.5043671403409637
2/      1.276110133041268
3/      1.292554254551292
4/      0.9940946653701164
5/      1.0227922786138604
6/      0.8829754269054131
7/      0.6630004108009008
8/      0.6114030741730833
9/      0.9952679444857384

D:\PowerConsumption\Pig\AverageConsumption>_

```

Pig Problem 3: Average Consumption

Description: Extracts the month component from the Date field, groups the dataset by month, and calculates the average active power consumption per month.

Pig Script:

```

data = LOAD 'file:///D:/PowerConsumption/data/household_power_consumption.csv'
    USING PigStorage(',')
    AS (
        Date:chararray,
        Time:chararray,
        Global_active_power:double,
        Global_reactive_power:double,
        Voltage:double,
        Global_intensity:double,
        Sub_metering_1:double,
        Sub_metering_2:double,
        Sub_metering_3:double
    );

```

```

clean_data = FILTER data BY Global_active_power IS NOT NULL;

with_month = FOREACH clean_data GENERATE
    SUBSTRING(Date, 3, 5) AS Month,
    Global_active_power;

grouped_data = GROUP with_month BY Month;

average_consumption = FOREACH grouped_data GENERATE
    group AS Month,
    AVG(with_month.Global_active_power) AS Average_Power;

STORE average_consumption
INTO 'file:///D:/PowerConsumption/Pig/output/AverageConsumption'
USING PigStorage('\t');

```

Execution & Output Verification:

```

D:\PowerConsumption\Pig\MaximumVoltage>type D:\PowerConsumption\Pig\output\MaximumVoltage\part-r-00000
254.15

D:\PowerConsumption\Pig\MaximumVoltage>

```

Pig Problem 4: High Power Filtering

Description: Filters out records where Global_active_power exceeds a high-consumption threshold of 5 kW.

Pig Script:

```

data = LOAD 'file:///D:/PowerConsumption/data/household_power_consumption.csv'
    USING PigStorage(',')
    AS (
        Date:chararray,
        Time:chararray,
        Global_active_power:double,
        Global_reactive_power:double,
        Voltage:double,
        Global_intensity:double,
        Sub_metering_1:double,
        Sub_metering_2:double,
        Sub_metering_3:double
    );

clean_data = FILTER data BY Global_active_power IS NOT NULL;
high_power = FILTER clean_data BY Global_active_power > 5;

STORE high_power
INTO 'file:///D:/PowerConsumption/Pig/output/HighPower'

```



```
USING PigStorage('\t');
```

Execution & Output Verification:

```
D:\PowerConsumption\Pig\HighPower>type D:\PowerConsumption\Pig\output\HighPower\part-m-00000
16/12/2006      17:25:00      5.36      0.436      233.63      23.0      0.0      1.0      16.0
16/12/2006      17:26:00      5.374      0.498      233.29      23.0      0.0      2.0      17.0
16/12/2006      17:27:00      5.388      0.502      233.74      23.0      0.0      1.0      17.0
16/12/2006      17:35:00      5.412      0.47      232.78      23.2      0.0      1.0      17.0
16/12/2006      17:36:00      5.224      0.478      232.99      22.4      0.0      1.0      16.0
16/12/2006      17:37:00      5.268      0.398      232.91      22.6      0.0      2.0      17.0
16/12/2006      17:44:00      5.894      0.0      232.69      25.4      0.0      0.0      16.0
16/12/2006      17:45:00      7.706      0.0      230.98      33.2      0.0      0.0      17.0
16/12/2006      17:46:00      7.026      0.0      232.21      30.6      0.0      0.0      16.0
16/12/2006      17:47:00      5.174      0.0      234.19      22.0      0.0      0.0      17.0
16/12/2006      18:05:00      6.052      0.192      232.93      26.2      0.0      37.0      17.0
16/12/2006      18:06:00      6.752      0.186      232.12      29.0      0.0      36.0      17.0
16/12/2006      18:07:00      6.474      0.144      231.85      27.8      0.0      37.0      16.0
16/12/2006      18:08:00      6.308      0.116      232.25      27.0      0.0      36.0      17.0
16/12/2006      18:35:00      6.072      0.0      232.48      26.4      0.0      27.0      17.0
17/12/2006      01:05:00      5.282      0.15      237.38      22.4      0.0      35.0      0.0
17/12/2006      01:09:00      5.242      0.178      239.49      21.8      0.0      37.0      0.0
17/12/2006      01:13:00      5.254      0.214      238.15      22.0      0.0      35.0      0.0
17/12/2006      01:14:00      5.246      0.23      237.91      22.0      0.0      36.0      0.0
17/12/2006      01:15:00      5.216      0.21      237.44      22.0      0.0      36.0      0.0
17/12/2006      09:02:00      5.908      0.134      237.9      25.4      0.0      38.0      0.0
17/12/2006      09:03:00      7.064      0.124      235.57      30.0      0.0      37.0      0.0
17/12/2006      09:04:00      6.926      0.118      235.35      29.4      0.0      37.0      0.0
17/12/2006      09:05:00      6.58      0.056      235.15      28.2      0.0      38.0      0.0
17/12/2006      09:06:00      6.05      0.0      235.22      26.2      0.0      37.0      0.0
17/12/2006      09:07:00      6.104      0.05      235.32      25.8      0.0      37.0      0.0
17/12/2006      10:35:00      5.776      0.266      232.89      24.8      37.0      36.0      17.0
17/12/2006      10:36:00      5.718      0.244      232.07      24.6      37.0      35.0      16.0
```

```
D:\PowerConsumption\Pig\SortConsumption>powershell -Command "Get-Content D:\PowerConsumption\Pig\output\HighPower\part-m-00001 -TotalCount 10"
2/3/2008      15:48:00      5.002      0.052      233.99      21.4      0.0      71.0      0.0
2/3/2008      15:49:00      5.13      0.0      233.68      21.8      0.0      72.0      0.0
2/3/2008      15:50:00      5.084      0.0      233.54      21.6      0.0      71.0      0.0
2/3/2008      15:51:00      5.098      0.062      233.97      21.8      0.0      71.0      0.0
2/3/2008      15:52:00      5.102      0.046      234.7      21.6      0.0      72.0      0.0
2/3/2008      15:53:00      5.126      0.064      234.94      21.8      0.0      72.0      0.0
2/3/2008      15:54:00      5.094      0.046      234.56      21.6      0.0      71.0      0.0
2/3/2008      15:55:00      5.072      0.06      233.82      21.6      0.0      72.0      0.0
4/3/2008      16:02:00      5.042      0.132      237.65      21.4      38.0      11.0      17.0
4/3/2008      16:03:00      6.306      0.1      235.45      26.8      37.0      32.0      17.0
```

```
D:\PowerConsumption\Pig\SortConsumption>powershell -Command "Get-Content D:\PowerConsumption\Pig\output\HighPower\part-m-00002 -TotalCount 10"
10/5/2009      16:01:00      5.002      0.088      238.46      21.0      1.0      74.0      1.0
10/5/2009      16:02:00      5.03      0.104      239.51      21.0      1.0      74.0      1.0
10/5/2009      16:03:00      5.04      0.124      239.62      21.0      1.0      75.0      0.0
10/5/2009      16:04:00      5.01      0.104      238.96      20.8      1.0      73.0      1.0
10/5/2009      16:05:00      5.032      0.12      239.1      21.0      1.0      75.0      1.0
10/5/2009      16:06:00      5.028      0.1      239.39      21.0      2.0      74.0      0.0
10/5/2009      16:07:00      5.056      0.12      239.63      21.0      1.0      75.0      1.0
10/5/2009      16:08:00      5.02      0.108      239.31      21.0      1.0      74.0      0.0
10/5/2009      16:09:00      5.026      0.158      239.78      20.8      1.0      75.0      1.0
10/5/2009      16:27:00      5.278      0.292      238.39      22.2      37.0      41.0      1.0
```

```
D:\PowerConsumption\Pig\SortConsumption>powershell -Command "Get-Content D:\PowerConsumption\Pig\output\HighPower\part-m-00003 -TotalCount 10"
25/7/2010      23:10:00      5.622      0.0      241.28      23.4      0.0      70.0      12.0
25/7/2010      23:11:00      5.916      0.0      240.55      24.6      0.0      75.0      19.0
25/7/2010      23:12:00      5.798      0.0      240.52      24.0      0.0      74.0      19.0
25/7/2010      23:13:00      5.844      0.07      240.6      24.2      0.0      75.0      19.0
25/7/2010      23:14:00      5.832      0.102      240.39      24.2      0.0      74.0      18.0
25/7/2010      23:15:00      5.798      0.082      239.97      24.0      0.0      74.0      19.0
27/8/2010      15:53:00      5.122      0.332      239.01      21.4      0.0      75.0      1.0
27/8/2010      15:54:00      5.062      0.312      238.94      21.2      0.0      74.0      1.0
27/8/2010      17:41:00      5.004      0.348      241.08      20.8      0.0      76.0      1.0
27/8/2010      17:44:00      5.33      0.446      239.47      22.4      0.0      75.0      6.0
```

Pig Problem 5: Sort by Power Consumption

Description: Performs a complete descending sort on the dataset according to Global_active_power levels.

Pig Script:

```
data = LOAD 'file:///D:/PowerConsumption/data/household_power_consumption.csv'
      USING PigStorage(',')
      AS (
        Date:chararray,
        Time:chararray,
        Global_active_power:double,
        Global_reactive_power:double,
        Voltage:double,
        Global_intensity:double,
        Sub_metering_1:double,
        Sub_metering_2:double,
        Sub_metering_3:double
      );

clean_data = FILTER data BY Global_active_power IS NOT NULL;
sorted_data = ORDER clean_data BY Global_active_power DESC;

STORE sorted_data
INTO 'file:///D:/PowerConsumption/Pig/output/SortConsumption'
USING PigStorage('\t');
```

Execution & Output Verification:

```
D:\PowerConsumption\Pig\SortConsumption>powershell -Command "Get-Content D:\PowerConsumption\Pig\output\SortConsumption\part-r-00000 -TotalCount 10"
```

22/2/2009	17:09:00	11.122	0.174	229.78	48.4	35.0	69.0	16.0
4/3/2007	19:34:00	10.67	0.516	230.2	46.4	37.0	69.0	16.0
4/3/2007	19:33:00	10.65	0.5	229.97	46.4	36.0	70.0	17.0
22/2/2009	17:08:00	10.536	0.192	230.24	45.8	23.0	69.0	16.0
30/11/2008	20:19:00	10.348	0.084	231.6	44.6	36.0	71.0	16.0
19/10/2008	01:24:00	10.29	0.302	230.9	44.6	35.0	66.0	17.0
27/1/2008	19:24:00	10.162	0.224	229.16	44.2	71.0	0.0	0.0
4/3/2007	19:32:00	10.154	0.502	229.72	44.4	36.0	61.0	16.0
30/11/2008	20:17:00	10.074	0.06	231.41	43.4	36.0	70.0	16.0
19/10/2008	01:25:00	10.064	0.324	231.48	43.4	36.0	69.0	17.0

Conclusion

This project successfully analyzed the Household Electric Power Consumption dataset using Hadoop MapReduce and Apache Pig. MapReduce was used to compute peak active power usage, average supply voltage levels, and total accumulated energy. Pig was used to find top high-load events, maximum system voltage spikes, monthly consumption trends, high-power filtering (>5 kW), and full dataset ordering. The analysis demonstrates how big data tools process high-frequency temporal power readings efficiently to generate actionable utility metrics.